

SmartCare v0.3 - Domain Model Workbook

Week 6 student resource

Requirement-to-Concept Trace

Requirement	Concept	State/behaviour	Decision
Staff can cancel an Appointment	Appointment	Appointment becomes deleted	Is the appointment currently booked and can it be cancelled?
Staff can book an appointment	Appointment	Appointment is booked and placed in calendar	Is the time slot and doctor free at that slot?
Staff can search patient by last name	Patient	System searches stored patient records and returns matching patient	Does a patient with that name exist?
The system produces operational reports.	Report	System generates a report based on the given data	What data should be included in report?
Practitioners can view their availability	Schedule	System displays available and booked time slots	Is the practitioner available at this selected time?
Client can book an appointment online	Appointment	Appointment is booked and placed in calendar	Is the time slot and doctor free at that slot?
Appointment is billed	Payment	Appointment is billed and patient is charged	Has the appointment been billed?
The system maintains patient records.	Patient	Patient information is stored and can be retrieved.	Does the patient record already exist?
Client details can be accessed in the appointment booking	Patient	Patient information is stored and can be retrieved in the appointment	Does the patient record already exist?

CRC Cards

Patient

Responsibilities	Collaborators
Store patient details	Appointment
Identify patient by last name	Appointment

Practitioner

Responsibilities	Collaborators
Store practitioner details	Appointment
View availability	Appointment

Appointment

Responsibilities	Collaborators
Store appointment details	Patient

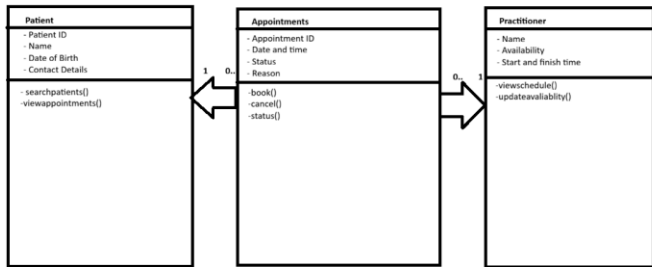
Send report to patient	Patient
------------------------	---------

Optional class

Responsibilities	Collaborators
Manage clients	Patient
Produce operational reports	Practitioner, patient

UML Class Diagram

Insert/draw UML here. Include defensible relationships and multiplicities.



Design Rationale

Explain class selection, responsibility allocation and key relationships.

The three classes selected are Patient, Practitioner and Appointment because they directly represent the core information that the SmartCare system needs to manage. The Patient class stores patient information and allows patient records to be identified and retrieved. The Practitioner class stores practitioner information and supports the management and viewing of practitioner availability and schedules.

The Appointment class represents a booking between a patient and a practitioner. It is responsible for storing appointment details, managing appointment status and allowing appointments to be cancelled while retaining their history.

Appointment acts as the link between Patient and Practitioner. Each appointment is associated with exactly one patient and one practitioner, while a patient and practitioner can each have zero or many appointments. This is represented using the multiplicities 1 and 0..*.

Responsibilities are allocated to the class that owns or directly manages the relevant information. For example, appointment status and cancellation are responsibilities of Appointment rather than Patient or Practitioner. This keeps the design simple and separates responsibilities between the classes.

The design intentionally avoids adding unnecessary classes because the case study specifies that the first version should be a simple, manageable application rather than a complex hospital information system

AI Design Review Record

AI suggestion	Evidence	Decision	Reason	Model change
Add an SMSReminder class to send appointment reminders.	The case study does not mention SMS reminders.	Reject	This is an unsupported feature and is not required for the initial system scope.	No change.
Add a Receptionist class to the UML diagram.	The case study describes manual processes but does not explicitly state that receptionists will use the system.	Do not add yet	The role should be validated with the client before being represented as a core class.	No change.
Add an Appointment class linking Patient and Practitioner.	The case study explicitly identifies appointment management as part of the initial system.	Accept	An appointment is necessary to represent bookings between patients and practitioners.	Added Appointment class and relationships to Patient and Practitioner.
Store cancelled appointments instead of deleting them.	The case study identifies a lack of reliable appointment history as an operational problem.	Accept	Retaining cancelled appointments supports reliable appointment history.	Added status to Appointment and cancellation behaviour.
Add an AI TreatmentRecommendation class.	No treatment recommendation functionality is described in the case study.	Reject	It is unsupported and outside the scope of a simple appointment booking system.	No change.